数学与 CS 问答笔记

answer-note

0101 规格评审：AF 的抽象值形状、RI 里哪条其实由类型保证、RI 没有维护者时会变成空话、防御性拷贝为什么是双向的。

November 03, 2026

Contents

1. 0101 规格评审：AF / RI / Safety 三处修正（Spec Review）｜问题 0101 3
   1.1. 费曼直觉 3
   1.2. 硬核机制 3
      1.2.1. AF 的形状：映射 vs 对的集合 3
      1.2.2. RI 里的三类子句 3
      1.2.3. 防御性拷贝是双向的 4
   1.3. 落到项目 4
   1.4. 陷阱与易错 4
   1.5. 关键点 5
   1.6. 变式检验 5
   1.7. 同类归类 5
   1.8. Go 语言机制补充 5
   1.9. 本次操作与当前状态 5
Index of Listings 7

1. 0101 规格评审：AF / RI / Safety 三处修正（Spec Review）｜问题 0101

1.1. 费曼直觉

学生交上来的三句话，每一句都踩在一个典型的边界上：

- **AF** 把「映射」写成了「对的集合」——听起来差不多，但抽象值的形状决定了你能不能用它推出 `Get` 的语义。
- **RI** 写了一条「键不重复」——这是 Go map 的类型保证，不是这份表示的性质。
- **Safety** 只写了出向拷贝，漏了入向——别名是双向的。

> 📌 **核心要点** 评审不是找错别字。每一处要问的都是同一个问题：**这句话约束的是谁、靠什么维持**。答不上来，就说明它放错位置了。

1.2. 硬核机制

1.2.1. AF 的形状：映射 vs 对的集合

抽象值必须是「键到值的映射」，理由很直接：`Get(k)` 的规格要求「返回 k 对应的值」。如果 AF 说抽象值是「键值对的集合」，那么「k 对应的值」这个概念在抽象层根本不存在——你只能回答「这对在不在集合里」。

AF 的写法	能推出 `Get` 吗	问题
`AF(mem) = { k -> mem[k] | ... }`	能：k 对应的值就是 mem[k]	需要覆盖空 map
`AF(mem)` = 键值对的集合	不能：没有「对应」这个概念	抽象值与客户端看到的行为不一致

1.2.2. RI 里的三类子句

子句类型	例子	靠什么维持
类型保证	map 的键不重复	Go 运行时与编译器，不需要你写
表示性质（真 RI）	键非空、值非空	实现自己：构造器与变值器
前置条件	调用者不能传空 key	调用者；实现可以不检查

> ⚠️ **陷阱预警** 最危险的状态不是「RI 写多了」，而是**写了 RI 却没有任何代码维持它**。比如写了「键非空」，但 `Set` 直接存，那这条 RI 在第一次调用时就被破坏了——后续所有推理都失效。

1.2.3. 防御性拷贝是双向的

```
// 入向：调用者的切片被存进 mem
func (kv *KV) Set(key, val []byte) (bool, error) {
  kv.mem[string(key)] = val
  return true, nil
}

// 调用者之后改自己的切片：
val[0] = 0x78   // mem 里存的值跟着变

// 出向：mem 里的切片被交给调用者
func (kv *KV) Get(key []byte) ([]byte, bool, error) {
  return kv.mem[string(key)], true, nil
}

// 调用者改返回的切片：mem 内部被改
```

Listing 1: 两条别名路径：入向（Set 存引用）与出向（Get 返引用）

1.3. 落到项目

本次交上来的三句与本项目的判定：

项	原文要点	判定	原因
AF	键唯一、值成对	🟨	抽象值应为映射；「键唯一」是类型保证；空 map 未覆盖
RI	键非空、键不重复、值非空	🟨	「键不重复」不是 RI；「键非空」缺维护者
Safety	mem 私有，Get 做防御性拷贝	🟩	方向对；漏 Set 的入向拷贝

📌 **核心要点** 按本人 2026-11-02 的约定：**强度跟随当前测试**。当前 `TestKVBasic` 没有空键、没有别名用例，所以这三处都可以先按「标注 + TODO(阶段末)」处理；但**写着却不实现**是不允许的——那会让规格与实现脱节。

1.4. 陷阱与易错

- **把类型保证写进 RI**：map 的键唯一、切片的长度可查——这些不是你的表示不变式。
- **AF 写成集合**：抽象值形状不对，`Get` 的语义就推不出来。
- **只防出向别名**：`Set` 存了调用者的切片，调用者之后改动会穿透进 `mem`。
- **RI 没有维护者**：写了「键非空」，`Set` 却不检查，等于没写。

⚠️ **陷阱预警** 零值 `KV{}` 的 `mem` 是 nil。`Open()` 之后才非 nil，所以「`mem != nil`」如果不写进 RI，就要接受 `KV{}` 直接 `Set` 会 panic（`assignment to entry in nil map`）。这是 Go 零值可用性的经典取舍。

1.5. 关键点

1. AF 的抽象值是**映射**，且必须覆盖空表示。
2. RI 里先剔除「类型保证」，剩下的才是你要维持的。
3. 每条 RI 都要回答「谁维持它」；答不上来就降级为前置条件或删掉。
4. 防御性拷贝分入向、出向两条路径。
5. 规格强度跟随测试，但规格与实现不能脱节。

1.6. 变式检验

把同一套判断搬到 `IntSet`（`elems []int`，升序无重复）：

- 「升序无重复」是表示性质（切片本身允许重复乱序）→ 真 RI，由 `Add` 维持；
- 「元素总数不超过容量」若实现从不检查 → 它是前置条件，不是 RI；
- 出向：`Elems()` 返回 `elems` 本身会让调用者改坏有序性；
- 入向：`Add(xs ...int)` 若直接 `append(elems, xs...)`，调用者之后改 `xs` 会穿透。

1.7. 同类归类

概念	6.102	6.031
C09 AF/RI	R07 Abstraction Functions and Rep Invariants	R11 Invariants
C07 别名	R05 散见	R08 Mutability and Immutability
C05 规格	R04 Specifications	R06 Specifications

1.8. Go 语言机制补充

```
// 零值 map 与空 map 是两件事
var a map[string][]byte   // nil：读安全，写 panic
b := map[string][]byte{} // 非 nil 空 map：读写都安全

// 防御性拷贝
go doc bytes.Clone
go doc slices.Clone
```

Listing 2: nil map 与空 map 的区别；标准库的拷贝工具

权威出处：`go doc bytes.Clone`、`go doc slices.Clone`、`go doc reflect.SliceHeader`（切片头三字段 Data / Len / Cap）。

1.9. 本次操作与当前状态

本次检查

- 交卷内容：`internal/db/kv.go` 的 AF / RI / Safety 三处注释（题 2–4），`Get` / `Set` / `Del` 仍是 `panic("TODO: 0101")`。
- 未交：题 1（红基线分类）、题 5（分区表）、题 6（实现）、题 7（验证）、题 8（内化 5 问）。
- 红基线现状：`Open` / `Close` 恢复为官方实现后，`assert.Nil` 通过，第一处 panic 在 `Set`（`kv.go:29`）。

评审结论

- AF 🟨：抽象值应为映射；「键唯一」属类型保证；空 map 未覆盖。
- RI 🟨：「键不重复」不是 RI；「键非空」「值非空」是真 RI 但缺维护者。
- Safety 🟩：出向拷贝写对，缺入向（`Set` 存引用）。

下一步

1. 改 AF 为映射写法并覆盖空 map；
2. RI 决定「键非空」的维护者（Set 检查 / 前置条件 / 删除）；
3. Safety 补入向拷贝一句；
4. 做题 5 分区表、题 6 实现，然后题 7 跑测试。

Index of Listings

Listing 1 两条别名路径：入向（Set 存引用）与出向（Get 返引用） 4
Listing 2 nil map 与空 map 的区别；标准库的拷贝工具 5